

ANSIBLE

MULTI-SERVER LAB

Vagrant



Ansible



Nginx



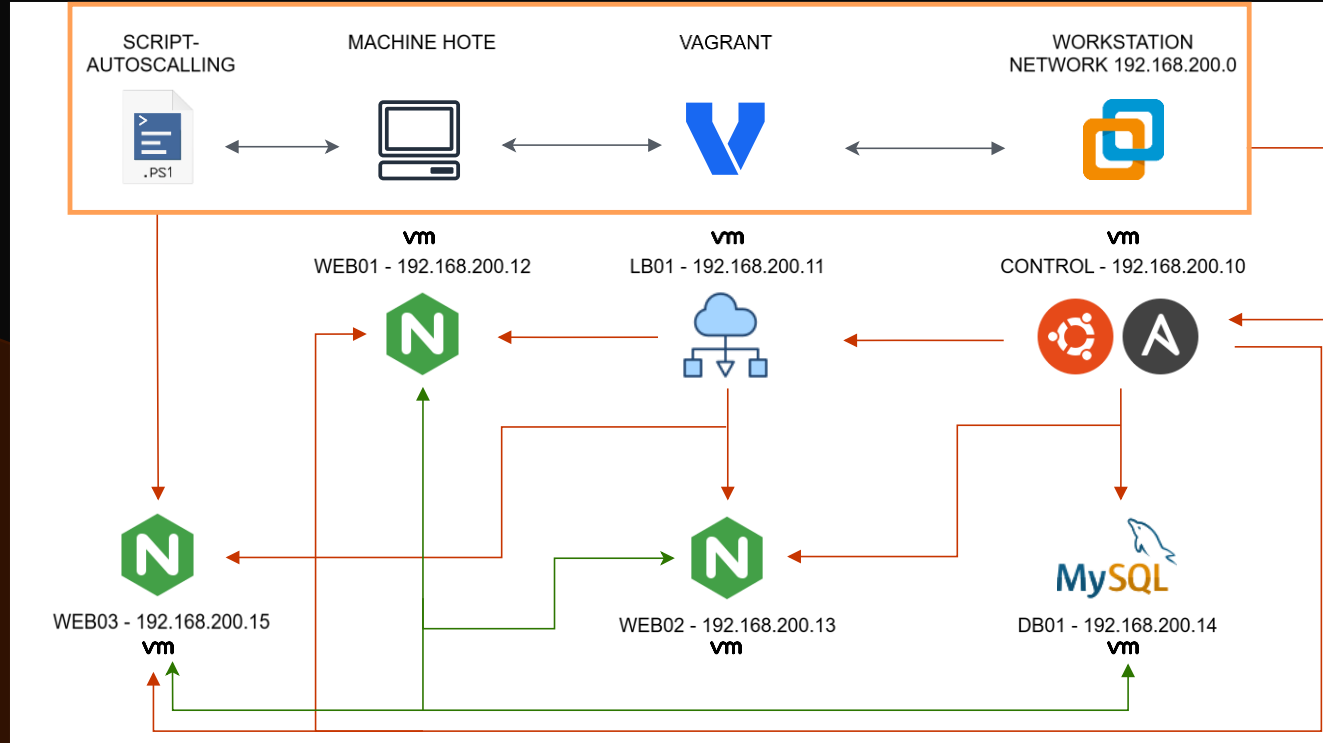
Vmware Workstation



Mysql



ARCHITECTURE



ANSIBLE PLAYBOOKS

Infrastructure configured from host control



01

nginx.yml

hosts: webservers

Install Nginx
Custom index page
Service enabled

02

loadbalancer.yml

hosts: loadbalancer

Nginx reverse proxy
Round-robin config
Default site removed

03

database.yml

hosts: database

MySQL install
Create appdb
Create appuser

04

security.yml

hosts: all

UFW firewall
Nginx sec headers
SSH hardening

AUTO-SCALING SIMULATION



PowerShell daemon monitors CPU

1

**Monitor
CPU**

web01 + web02
every 30s

2

**CPU > 80%
3 checks**

90 secondes
consecutives

3

**Scale UP
web03**

vagrant up
+ Ansible

4

**CPU < 20%
3 checks**

charge basse
consecutive

5

**Scale DOWN
web03**

vagrant halt
+ lb update

30s

check interval

3x

checks before action

~2min

scale-up time

100%

CPU threshold up

VAGRANT DEPLOYMENT



"In the Vagrantfile, we provision our resources according to our hardware requirements. (You can find the file in my Git repository.)

```
PS C:\Users\rabbitSix\Desktop\ansible-multiserver-lab> vagrant up control
Bringing machine 'control' up with 'vmware_desktop' provider...
==> control: Checking if box 'bento/ubuntu-22.04' version '202510.26.0' is up to date..
==> control: Verifying vmnet devices are healthy...
==> control: Preparing network adapters...
==> control: Starting the VMware VM...
==> control: Waiting for the VM to receive an address...
==> control: Forwarding ports...
control: -- 22 => 2222
==> control: Waiting for machine to boot. This may take a few minutes...
control: SSH address: 127.0.0.1:2222
control: SSH username: vagrant
control: SSH auth method: private key
control:
control: Vagrant insecure key detected. Vagrant will automatically replace
control: this with a newly generated keypair for better security.
control:
control: Inserting generated public key within guest...
control: Removing insecure key from the guest if it's present...
control: Key inserted! Disconnecting and reconnecting using new SSH key...
==> control: Machine booted and ready!
==> control: Setting hostname...
```

"Here, for example, we brought up our 'control' VM with its configuration as described in the Vagrantfile, and used the same command to bring up the other VMs."

VAGRANT DEPLOYMENT



Next, connect to our VM via SSH using the command `vagrant ssh` "hostname". In this case, the name is 'control'.

```
PS C:\Users\rabbitSix\Desktop\ansible-multiserver-lab> vagrant ssh control
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-160-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Sat Jun 27 05:41:20 AM UTC 2026

System load:  0.06               Processes:            227
Usage of /:   18.1% of 30.34GB   Users logged in:     0
Memory usage: 32%               IPv4 address for eth0: 192.168.253.144
Swap usage:   0%

This system is built by the Bento project by Chef Software
More information can be found at https://github.com/chef/bento

Use of this system is acceptance of the OS vendor EULA and License Agreements.
vagrant@control:~$ ansible --version
ansible 2.10.8
  config file = None
  configured module search path = ['/home/vagrant/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python3/dist-packages/ansible
  executable location = /usr/bin/ansible
  python version = 3.10.12 (main, Aug 15 2025, 14:32:43) [GCC 11.4.0]
```

"Once connected to the 'control' Ubuntu host, we verify the Ansible installation (specified in the Vagrantfile provisioning), which enables us to manage the other VMs."

VAGRANT DEPLOYMENT



Next, we check the status of our Vagrant VMs using the "**vagrant status**" command.

```
C:\Users\rabbitSix\Desktop\ansible-multiserver-lab>vagrant status
```

```
Current machine states:
```

control	running (vmware_desktop)
lb01	running (vmware_desktop)
web01	running (vmware_desktop)
web02	running (vmware_desktop)
db01	running (vmware_desktop)

```
This environment represents multiple VMs. The VMs are all listed  
above with their current state. For more information about a specific  
VM, run `vagrant status NAME`.
```

"Here, we can see that all our VMs have been created and are in a **running status**."

ANSIBLE DEPLOYMENT



Create and edit **hosts** using vim or nano at the following path:
/etc/ansible/.

```
vagrant@control:/etc/ansible$ ls
autoscale.sh  hosts  playbooks
vagrant@control:/etc/ansible$ cat hosts
[loadbalancer]
lb01 ansible_host=192.168.200.11

[webservers]
web01 ansible_host=192.168.200.12
web02 ansible_host=192.168.200.13
web03 ansible_host=192.168.200.15

[database]
db01 ansible_host=192.168.200.14

[all:vars]
ansible_user=vagrant
ansible_ssh_private_key_file=/home/vagrant/.ssh/id_rsa
ansible_ssh_common_args='-o StrictHostKeyChecking=no'
vagrant@control:/etc/ansible$ _
```

"This hosts file is where we define all the hosts we need to manage. It contains the IP addresses of each server, their specific hostnames, and the SSH key information required for communication between 'control' and the other servers."

ANSIBLE DEPLOYMENT



After the initial setup, we verify that our VMs can communicate properly. We use the standard ping command to test network connectivity, and `ansible all -m ping` to confirm that SSH access and Ansible authentication are working perfectly across all hosts.

```
vagrant@control:/etc/ansible$ ansible all -m ping
web02 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
db01 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
lb01 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
web01 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
```

```
vagrant@control:~$ ping 192.168.200.11
PING 192.168.200.11 (192.168.200.11) 56(84) bytes of data.
64 bytes from 192.168.200.11: icmp_seq=1 ttl=64 time=0.382 ms
64 bytes from 192.168.200.11: icmp_seq=2 ttl=64 time=0.621 ms
^C
--- 192.168.200.11 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1007ms
rtt min/avg/max/mdev = 0.382/0.501/0.621/0.119 ms
vagrant@control:~$ ping 192.168.200.12
PING 192.168.200.12 (192.168.200.12) 56(84) bytes of data.
64 bytes from 192.168.200.12: icmp_seq=1 ttl=64 time=0.558 ms
64 bytes from 192.168.200.12: icmp_seq=2 ttl=64 time=0.217 ms
^C
--- 192.168.200.12 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1016ms
rtt min/avg/max/mdev = 0.217/0.387/0.558/0.170 ms
vagrant@control:~$ ping 192.168.200.13
PING 192.168.200.13 (192.168.200.13) 56(84) bytes of data.
64 bytes from 192.168.200.13: icmp_seq=1 ttl=64 time=0.951 ms
64 bytes from 192.168.200.13: icmp_seq=2 ttl=64 time=0.340 ms
^C
--- 192.168.200.13 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 999ms
rtt min/avg/max/mdev = 0.340/0.645/0.951/0.305 ms
vagrant@control:~$ ping 192.168.200.14
PING 192.168.200.14 (192.168.200.14) 56(84) bytes of data.
64 bytes from 192.168.200.14: icmp_seq=1 ttl=64 time=0.565 ms
64 bytes from 192.168.200.14: icmp_seq=2 ttl=64 time=0.395 ms
^C
--- 192.168.200.14 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1029ms
rtt min/avg/max/mdev = 0.395/0.480/0.565/0.085 ms
```

ANSIBLE DEPLOYMENT



Create and edit the playbooks using vim or nano at the following path:
`/etc/ansible/playbooks`.

```
vagrant@control:/etc/ansible/playbooks$ ls  
database.yml  loadbalancer.yml  nginx.yml  security.yml  
vagrant@control:/etc/ansible/playbooks$ _
```

Here, each playbook has its specific role: `database.yml`, for example, configures our db01 VM and installs the MySQL relational database engine; `loadbalancer.yml` balances workloads across our application servers; `nginx.yml` configures our web servers; and `security.yml` manages our environment's security policies

ANSIBLE DEPLOYMENT



To apply our configurations from the control host, simply run: **ansible-playbook <playbook>.yaml**.

```
vagrant@control:/etc/ansible/playbooks$ sudo nano -l loadbalancer.yml
vagrant@control:/etc/ansible/playbooks$ ansible-playbook loadbalancer.yml

PLAY [install and configure nginx load balancer] *****
*

TASK [Gathering Facts] *****
*
ok: [lb01]

TASK [Update apt cache] *****
*
ok: [lb01]

TASK [install nginx] *****
*
ok: [lb01]

TASK [config nginx as Loadbalancer] *****
*
ok: [lb01]

TASK [enable load balancer config] *****
*
changed: [lb01]

TASK [removed default nginx site] *****
*
changed: [lb01]

TASK [restart nginx] *****
*
changed: [lb01]

PLAY RECAP *****
*
lb01          : ok=7    changed=3    unreachable=0    failed=0
skipped=0     rescued=0    ignored=0
```

Run the command for each
playbook, ensuring the output
shows that all tasks are 'ok'.

TEST OF OUR SERVICES



Our Nginx servers is up

```
vagrant@control:/etc/ansible/playbooks$ curl http://192.168.200.12
<html>
  <body>
    <h1> hello from web01</h1>
  </body>
</html>
vagrant@control:/etc/ansible/playbooks$ curl http://192.168.200.13
<html>
  <body>
    <h1> hello from web02</h1>
  </body>
</html>
vagrant@control:/etc/ansible/playbooks$ _
```

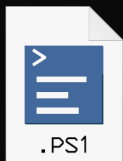
TEST OF OUR SERVICES



Our Loadbalancer is up

```
vagrant@control:/etc/ansible/playbooks$ curl http://192.168.200.11
<html>
  <body>
    <h1> hello from web02</h1>
  </body>
</html>
vagrant@control:/etc/ansible/playbooks$ curl http://192.168.200.11
<html>
  <body>
    <h1> hello from web01</h1>
  </body>
</html>
vagrant@control:/etc/ansible/playbooks$ curl http://192.168.200.11
<html>
  <body>
    <h1> hello from web02</h1>
  </body>
</html>
vagrant@control:/etc/ansible/playbooks$ curl http://192.168.200.11
<html>
  <body>
    <h1> hello from web01</h1>
  </body>
</html>
vagrant@control:/etc/ansible/playbooks$ curl http://192.168.200.11
<html>
  <body>
    <h1> hello from web02</h1>
  </body>
</html>
```

POWERSHELL AUTOSCALLING SCRIPT



Ce script PowerShell automatise l'**autoscaling** (le dimensionnement dynamique) de l'infrastructure. Il surveille en continu la charge CPU moyenne des serveurs web principaux (web01 et web02) toutes les 30 secondes :

- **Si la charge dépasse 80%** pendant 3 vérifications, il déploie une troisième VM (web03) via Vagrant, la configure avec Ansible et l'ajoute au répartiteur de charge Nginx pour absorber le trafic (**Scale-Up**).
- **Si la charge chute sous 20%**, il retire proprement web03 du répartiteur de charge Nginx et l'éteint pour libérer les ressources de l'infrastructure (**Scale-Down**).

POWERSHELL AUTOSCALLING SCRIPT TEST



Installation of the stress package to simulate a heavy workload on the server side, followed by using this tool to push the server's CPU usage to a high level for several minutes.

```
vagrant@web01:~$ sudo apt install stress -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
stress is already the newest version (1.0.5-1).
0 upgraded, 0 newly installed, 0 to remove and 153 not upgraded.
vagrant@web01:~$ stress --cpu 4 --timeout 120
stress: info: [8289] dispatching hogs: 4 cpu, 0 io, 0 vm, 0 hdd
stress: info: [8289] successful run completed in 123s
vagrant@web01:~$ _
```

POWERSHELL AUTOSCALLING SCRIPT TEST



```
PS C:\Users\rabbitSix\Desktop\ansible-multiserver-lab> .\autoscale.ps1
[06/29/2026 02:58:13] Autoscaler démarre
[06/29/2026 02:58:15] CPU web01=5% web02=4% avg=4.5%
[06/29/2026 02:58:15] CPU bas check 1/3
[06/29/2026 02:58:48] CPU web01=25% web02=25% avg=25%
[06/29/2026 02:59:21] CPU web01=58% web02=55% avg=56.5%
[06/29/2026 02:59:54] CPU web01=75% web02=75% avg=75%
[06/29/2026 03:00:27] CPU web01=86% web02=85% avg=85.5%
[06/29/2026 03:00:27] CPU eleve check 1/3
[06/29/2026 03:01:00] CPU web01=92% web02=97% avg=94.5%
[06/29/2026 03:01:00] CPU eleve check 2/3
[06/29/2026 03:01:33] CPU web01=95% web02=98% avg=96.5%
[06/29/2026 03:01:33] CPU eleve check 3/3
[06/29/2026 03:01:33] SCALE UP - lancement web03
```

In this Logs :

- **02:58:** System starts in a normal state with a low CPU load of **4.5%**.
- **02:59:** A heavy workload is applied, pushing the average CPU usage up to **75%**.
- **03:00:** CPU crosses the **80%** threshold, hitting **85.5%** (Check 1/3).
- **03:01:** CPU sustains high stress at **94.5%** (Check 2/3) and **96.5%** (Check 3/3).
- **03:01 (Action):** Having met the 3 consecutive checks requirement, the script autonomously triggers **SCALE UP** to deploy web03.

POWERSHELL AUTOSCALLING SCRIPT TEST

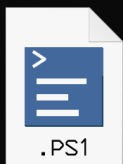


In this logs :

the automated **SCALE UP** trigger initiating the deployment of web03. Vagrant takes over to clone the Ubuntu 22.04 base image using the VMware provider, automatically configures the network adapters, and resolves an SSH port collision by remapping host port 22 to **2204** before successfully booting up the virtual machine.

```
[06/29/2026 03:01:33] CIO CIOV CHECK 3/3
[06/29/2026 03:01:33] SCALE UP - lancement web03
Bringing machine 'web03' up with 'vmware_desktop' provider...
==> web03: Cloning VMware VM: 'bento/ubuntu-22.04'. This can take some time...
==> web03: Checking if box 'bento/ubuntu-22.04' version '202510.26.0' is up to date...
==> web03: Verifying vmnet devices are healthy...
==> web03: Preparing network adapters...
==> web03: Fixed port collision for 22 => 2222. Now on port 2204.
==> web03: Starting the VMware VM...
==> web03: Waiting for the VM to receive an address...
==> web03: Forwarding ports...
web03: -- 22 => 2204
==> web03: Waiting for machine to boot. This may take a few minutes...
```

POWERSHELL AUTOSCALLING SCRIPT TEST



The successful **SCALE DOWN** sequence after the simulated traffic spike subsides. Once the workload cools off, the average CPU load falls below the 20% threshold, hitting 15%, 8.5%, and 9% across three consecutive 30-second checks.

```
than running sudo
[06/29/2026 03:22:27] web03 ajoute au pool
[06/29/2026 03:23:00] CPU web01=10% web02=39% avg=24.5%
[06/29/2026 03:23:33] CPU web01=6% web02=24% avg=15%
[06/29/2026 03:23:33] CPU bas check 1/3
[06/29/2026 03:24:04] CPU web01=3% web02=14% avg=8.5%
[06/29/2026 03:24:04] CPU bas check 2/3
[06/29/2026 03:24:36] CPU web01=9% web02=9% avg=9%
[06/29/2026 03:24:36] CPU bas check 3/3
[06/29/2026 03:24:36] SCALE DOWN - arret web03
lb01 | CHANGED | rc=0 >>
upstream webservers { server 192.168.200.12; server 192.168.200.13; }
server { listen 80; location / { proxy_pass http://webservers; } }
[WARNING]: Consider using 'become', 'become_method', and 'become_user' rather
than running sudo
==> web03: Attempting graceful shutdown of VM...
```

Upon the third confirmation at 03:24:36, the script triggers the downscaling mechanism: it immediately rewrites the Nginx load balancer configuration to remove 192.168.200.15 (web03), restarts the proxy service, and safely initiates a graceful shutdown of the now-redundant web03 virtual machine.

The full source code, playbooks, and scripts for this project can be found on my GitHub repository.

THANK YOU